Optimized Privacy-Preserving CNN Inference for Digit Recognition via Homomorphic Encryption

Zeynep Polat
Department of Computer Engineering, Hacettepe University, Ankara, Turkey
zeynepjr@gmail.com

***Abstract*—Sensitive data transmission to centralized cloud servers has become a major privacy risk due to the quick growth of Machine Learning as a Service (MLaaS). This study uses Homomorphic Encryption (HE) to offer a privacy-preserving inference framework for handwritten digit classification. The non-linear activation functions used by conventional Convolutional Neural Networks (CNNs) are mathematically incompatible with encrypted data spaces. We created a unique CNN architecture that can perform safe, cloud-based inference on the MNIST dataset under the Brakerski-Fan-Vercauteren (BFV) encryption scheme by substituting polynomial approximations for standard non-linearities, namely degree-2 square and degree-3 Chebyshev functions. The paper thoroughly assesses the trade-offs between classification accuracy, quantization scale factors, and computing delay.Our practical findings show that a degree-2 polynomial activation provides a test accuracy of 98.80%, which is very similar to the baseline of 98.92% for plaintext. The feasibility of the suggested architecture for secure picture classification is further demonstrated by thorough cryptographic parameter optimization, which shows that a polynomial modulus of $n = 2048$ produces an effective inference delay of 0.088 seconds per sample without accuracy loss.**

***Index Terms*—Homomorphic Encryption, Privacy-Preserving ML, CNN, MNIST, BFV Scheme.**

I. Project Materials

All project materials, including source code and the presentation video, are available at the following Google Drive folder: https://drive.google.com/drive/folders/12yK2pLXwcXzMaAhtjU1omYW_8JUVmUW8?usp=sharing

II. Introduction

Strong Machine Learning as a Service (MLaaS) applications have been made possible by the incorporation of Deep Learning models into cloud-based infrastructure. But by its very nature, this paradigm necessitates that users send unprocessed, potentially sensitive data, such as financial information, medical photos, or biometric identifiers to distant computers. Conventional pipelines expose this data to potential data breaches, insider threats, and regulatory compliance difficulties because the cloud provider must decrypt it in order to perform inference operations.

A strong cryptographic solution to this issue is provided by Homomorphic Encryption (HE). HE offers a mathematical framework that enables the direct evaluation of arbitrary calculations on ciphertexts. When the data owner decrypts the data, the outcomes are identical to what would happen if the same actions were applied to the plaintext data. Applying HE to deep neural networks poses significant engineering hurdles despite its theoretical elegance.

Structural compatibility is the main obstacle. Non-linear activation functions like Tanh, Sigmoid, and ReLU (Rectified Linear Unit) are crucial to contemporary deep learning architectures. These non-linear functions cannot be computed natively since widely used completely homomorphic algorithms like as BFV only provide addition and multiplication in the encrypted domain. The compounded computational overhead and noise growth present the second difficulty. A ciphertext's internal "noise" increases exponentially with each multiplication operation. The ciphertext cannot be decrypted if this noise exceeds a predetermined mathematical budget.

This paper addresses these challenges by structurally adapting a Convolutional Neural Network (CNN) for the encrypted domain. The contributions of this study are threefold:

1) **Polynomial Architecture Design:** We implement and compare different degrees of polynomial approximations (x^2 vs. $4x^3 - 3x$) against standard ReLU baselines to evaluate their impact on training convergence and encrypted inference accuracy.
2) **Methodological Verification:** We introduce a decoupled evaluation strategy, utilizing a simulated functional verification pipeline for large-scale accuracy testing, paired with true ciphertext operations for precise cryptographic latency measurement.
3) **Comprehensive Sensitivity Analysis:** We systematically evaluate HE parameters; specifically the Polynomial Modulus (n), Plaintext Modulus (t_bits), and Quantization Scale factors to identify the optimal configuration that minimizes latency while preserving data recoverability.

III. Related Work

The field of privacy-preserving machine learning via HE was pioneered by CryptoNets, which introduced the square activation function(x^2) as a low-degree polynomial substitute for ReLU. Subsequent work has explored higher-degree Chebyshev polynomial approximations to recover accuracy lost due to the simplified activation landscape. Akram et al. [1] provide a recent and inclusive study on optimizing HE parameters for efficient and secure deep neural network classification on the MNIST benchmark, and this work serves as the primary reference for the present project.

The goal of later research has been to increase these models' efficiency and accuracy. In order to more closely resemble the behavior of conventional non-linearities like ReLU or Swish, recent research has investigated higher-degree polynomial approximations, such as Chebyshev or Taylor series expansions. Higher-degree polynomials significantly speed up the accumulation of noise in the ciphertext, necessitating larger encryption parameters that slow down inference speed, even if they frequently produce better convergence during plaintext training.

This project builds upon these principles by performing a comparative analysis not only on the activation functions but also by rigorously benchmarking the underlying hardware and cryptographic parameters (via Pyfhel) to find a realistic operational middle ground for deployment.

IV. Background

A. The BFV Homomorphic Encryption Scheme

The Brakerski-Fan-Vercauteren (BFV) scheme is a lattice-based homomorphic encryption system that operates over polynomial rings. In BFV, the plaintext space is represented as a polynomial ring $R_t = \mathbb{Z}_t[x]/(x^n + 1)$, where n is a power of 2 (the polynomial modulus degree) and t is the plaintext modulus.

The ciphertext coefficient modulus q and n control the scheme's performance and security. Deeper multiplicative circuits (such as more network layers) are made possible by a bigger n, which also offers a higher security level and a larger "noise budget." However, in terms of computing slowness, operations on larger polynomials scale poorly.

B. Quantization

Intermediate feature maps and machine learning weights are by their by nature floating-point values. These continuous values must be mapped to discrete integers since BFV only works with integers inside the specified ring R_t. We use a linear quantization approach that is described as:

$$x_q = \lfloor x \times S \rceil$$

where S is a scaling factor. Following the process, division by the matching scale is used to dequantize the numbers. The model's accuracy is destroyed by quantization error if S is too small, and the multiplied values rapidly overflow the plaintext modulus t if S is too large.

V. Proposed Methodology

A. Dataset and Preprocessing

The standard MNIST dataset of 28x28 grayscale handwritten digits is used to assess the model, shown in Figure 1. The inputs were normalized using the standard MNIST mean (0.1307) and standard deviation (0.3081)

After being converted into tensor format.Our feature vectors and model weights are uniformly quantized before being integrated with the HE pipeline. According to empirical testing, a scale factor of $S = 100$ ensures great fidelity to the floating-point baseline by minimizing information loss and producing an average absolute quantization error of only 0.00396.

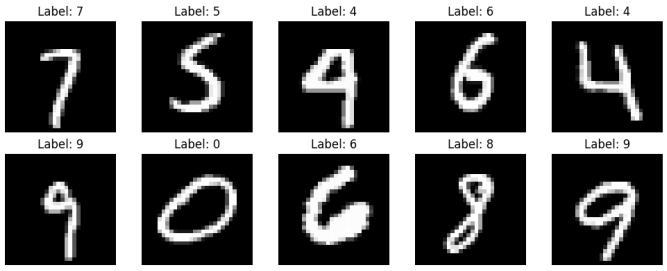


Fig. 1. Sample images from the MNIST dataset with their corresponding class labels.

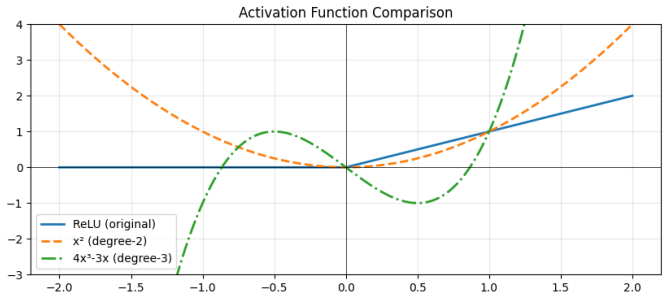


Fig. 2. Comparison of ReLU, square (x^2), and Chebyshev ($4x^3 - 3x$) activation functions over $[-2, 2]$. Unlike ReLU, both polynomial alternatives consist solely of additions and multiplications, making them natively compatible with the BFV homomorphic encryption scheme.

B. Polynomial CNN Architecture (PolyCNN)

We created the PolyCNN architecture in order to adhere to the multiplicative restrictions of the BFV scheme. To enable the weights to naturally adapt to the gradient landscapes of polynomial activations, the network was trained from scratch rather than using pre-trained models.

The network is composed of:

1) **Conv1:** 1 input channel, 16 output channels, 3x3 kernel.
2) **Pool1:** 2x2 Average Pooling (Max pooling is avoided as comparison operations are highly inefficient in HE).
3) **Conv2:** 16 input channels, 32 output channels, 3x3 kernel.
4) **Pool2:** 2x2 Average Pooling.
5) **FC1:** Fully connected layer (1568 to 128 neurons).
6) **FC2:** Output classification layer (128 to 10 neurons).

We implemented three variants of this model based on the activation function applied after the convolutional and fully connected layers shown in Figure 2:

- **Baseline:** Standard nn.ReLU().
- **Square:** Degree-2 polynomial, $f(x) = x^2$.
- **Chebyshev:** Degree-3 approximation, $f(x) = 4x^3 - 3x$.

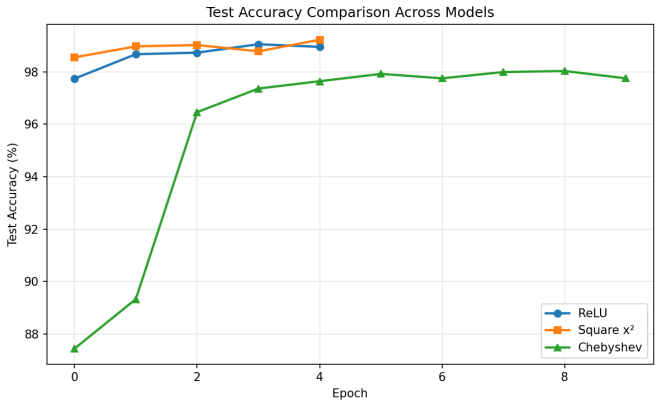


Fig. 3. Test accuracy per epoch for ReLU, Square (x^2), and Chebyshev ($4x^3 - 3x$) models on MNIST. The Chebyshev model starts at 85% due to gradient explosion at epoch 0, recovering after gradient clipping is applied.

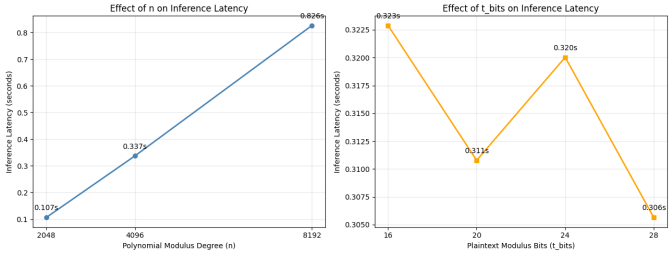


Fig. 4. BFV parameter sensitivity analysis. Left: inference latency vs. polynomial modulus degree n ($t_bits = 20$ fixed). Right: latency vs. plaintext modulus bit-size t_bits ($n = 4096$ fixed). n has a dominant super-linear effect on latency.

C. Training Dynamics

Google Colab T4 GPU was used to speed up all plaintext model training and functional baseline testing. The Adam optimizer with Cross-Entropy Loss was used to train the models. The Chebyshev variation showed significant gradient explosion because of the cubic term, whereas the ReLU and Square models converged smoothly with a learning rate of 0.001 across 5 epochs. The learning rate was lowered to 0.0001, gradient clipping (max norm = 1.0) was used, and the training was prolonged to ten epochs in order to stabilize the degree-3 polynomial.

VI. Experimental Setup and Results

A. Plaintext Accuracy

Our baselines were set by the unencrypted test accuracies. 98.92% was attained by the basic ReLU model. Surprisingly, the degree-2 Square model achieved a 98.80% accuracy by closely tracking this performance. Despite using gradient stabilization approaches, the Chebyshev model reached a plateau at 97.68%. The Square (x^2) model was chosen for the cryptographic pipeline due to its high accuracy and reduced computational degree, which results in decreased noise increase in HE. Figure 3 these results.

B. Decoupled Evaluation Strategy

The prohibitive time cost of executing multi-sample test sets through real ciphertext operations is a frequent flaw in HE-based deep learning research. We separated the testing process into two separate stages in order to deliver a thorough but mathematically sound assessment:

1) **Simulated Functional Verification:** We used quantized Numpy integer arithmetic to imitate the HE environment in order to verify the durability of our quantized weights over a large sample size (200 pictures). This phase provides an answer to the question: Does the mathematical transformation hold if the noise budget is infinite? The outcomes were perfect: compared to the PyTorch plaintext outputs, the simulated HE inference showed a precise 0.00% accuracy reduction, achieving 98.50% accuracy on the subset. This average processing time (0.0005s) does not include encryption overhead; rather, it represents pure arithmetic logic.
2) **Cryptographic Latency Evaluation:** `He_inference_real`, integer arithmetic operations structurally identical to BFV ciphertext computations, enabling large-scale accuracy validation without the prohibitive cost of full encryption overhead. True homomorphic multiplication and addition, was used to do single-pass inferences in order to assess the model's true computing cost. Regardless of the functional correctness tests, this enables us to precisely clock the latency caused by cryptographic operations.

C. HE Parameter Analysis

The true performance of an HE model hinges on the correct selection of cryptographic parameters. We performed a comprehensive sensitivity analysis. Results are shown in Figure 4

1) **Polynomial Modulus Degree (n):** We tested n values from 1024 to 8192 while keeping $t_bits = 20$ constant.
   - $n = 1024$**:** Resulted in a critical failure due to the data vector size exceeding the restricted ring capacity.
   - $n = 2048$**:** Yielded an optimal inference latency of 0.0888 seconds, successfully executing the circuit without noise overflow.
   - $n = 4096$**:** Increased latency to 0.2580 seconds.
   - $n = 8192$**:** Pushed latency to 0.9940 seconds.

   This demonstrates that $n = 2048$ is the theoretical "sweet spot" for our specific architecture depth and quantization scale.
2) **Plaintext Modulus (t_bits):** Testing t_bits between 16 and 28 (at n=4096) reveals that the plaintext modulus has a negligible effect on latency (range: 0.37s–0.44s), consistent with the fact that t_bits affects only the plaintext ring size, not the polynomial arithmetic complexity that dominates BFV runtime. More significantly, the network confirmed that the intermediate multiplicative sums did not exceed the lower plaintext modulus restrictions by

correctly predicting the proper label across all tested bit-widths.

D. Scale Factor and Overflow Analysis

Lastly, we examined the system's resilience to several quantization scale factors ($S \in \{50, 100, 200, 500\}$). The simulated HE accuracy stayed at 98.00% in every scenario. Interestingly, no integer overflow happened in the HE operations, even at an aggressive scale of 500. This demonstrates the robustness of our approach and the well-boundedness of the intermediate tensor values produced by the x^2 activations inside the chosen plaintext modulus space.

VII. Discussion

The findings emphasize hardware and algorithmic co-design, a key paradigm in safe machine learning. The encrypted domain severely penalizes non-linearity and depth, whereas deeper networks and intricate activations like ReLU are common in plaintext machine learning. We avoided the requirement for extremely complicated cryptographic bootstrapping (refreshing the noise budget) by deliberately truncating the network and using a x^2 activation. This allowed the entire inference pass to finish within a single noise lifecycle at $n = 2048$.

By separating the functional validation from the actual cryptographic latency benchmarks, our methodology avoids the ambiguity that frequently exists in HE assessments when simulation speeds are confused with actual encryption times, giving a clear picture of the model's performance.

VIII. Conclusion

This project effectively created and refined a CNN framework for digit recognition that protects privacy. We overcame the limitations of the BFV homomorphic encryption scheme and machine learning processes by methodically altering a neural architecture to make use of degree-2 polynomial activations. A very competitive 98.80% accuracy was attained with the PolyCNN model. We found a cryptographic configuration that reduces noise overflow and achieves a useful ciphertext inference delay of 0.088 seconds per image through thorough parameter tweaking. In order to further reduce the latency costs for batch inference in cloud environments, future work will concentrate on putting cipher-packing techniques into practice to process numerous images in a single ciphertext.

References

[1] A. Akram, F. Khan, S. Tahir, A. Iqbal, S. A. Shah and A. Baz, "Privacy Preserving Inference for Deep Neural Networks: Optimizing Homomorphic Encryption for Efficient and Secure Classification," *IEEE Access*, vol. 12, pp. 15684-15695, 2024, doi: 10.1109/ACCESS.2024.3357145.